-- 以下为AI生成的图文笔记内容 --

一、*Github Action基础概念00:00*

1. Github Action 00:04

-

- **定义**: GitHub的自动化流水线，通过编写action脚本指定GitHub完成一系列自动化操作
- **核心功能**:
  - 实现CI/CD（持续集成/持续交付）流水线
  - 可作为免费云服务器使用
  - 支持各种自定义自动化任务

1）自动化流水线示例 00:16

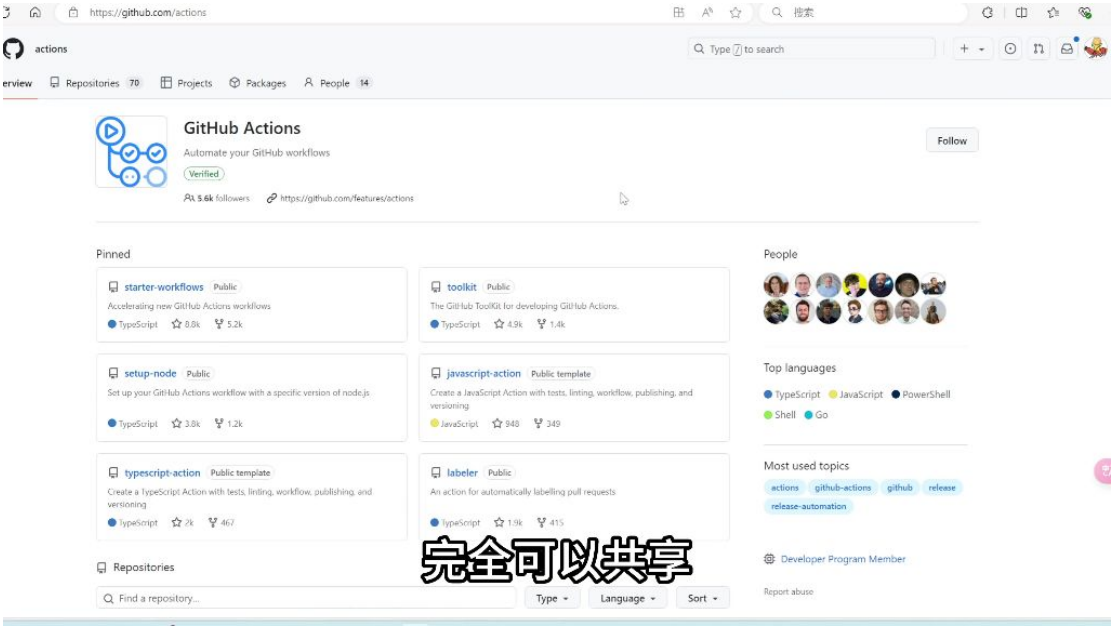
自动化流水线

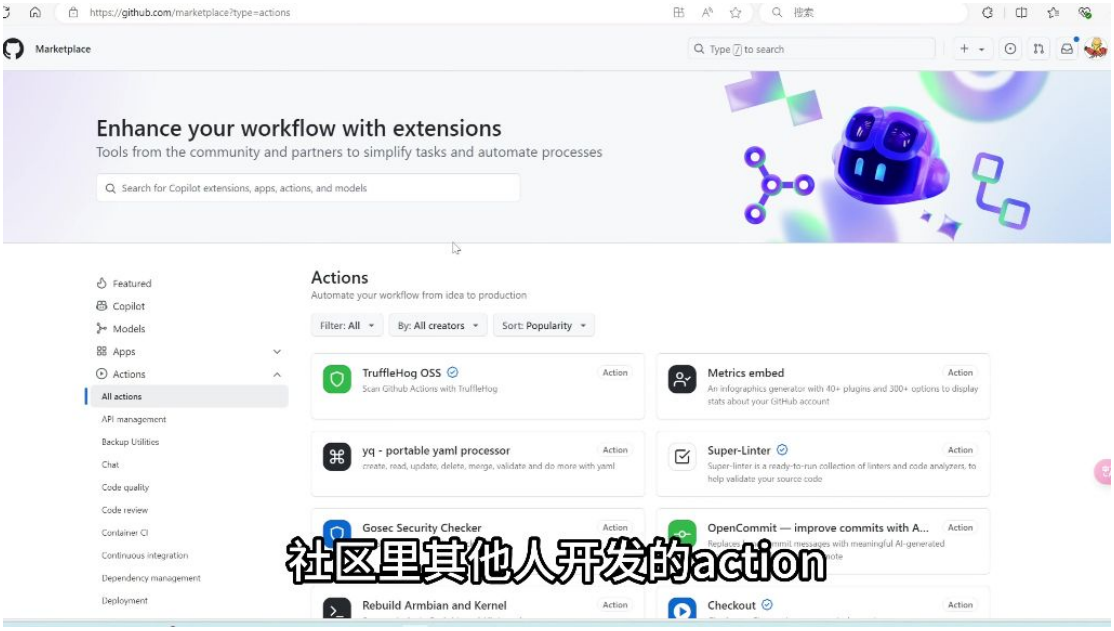
推送到服务器并且完成部署

-
- **典型流程**:
  - 代码提交触发自动化流程
  - 自动执行单元测试
  - 自动完成代码编译
  - 程序构建并推送到服务器
  - 自动完成部署
- **优势**: 实现从代码提交到部署的全流程自动化

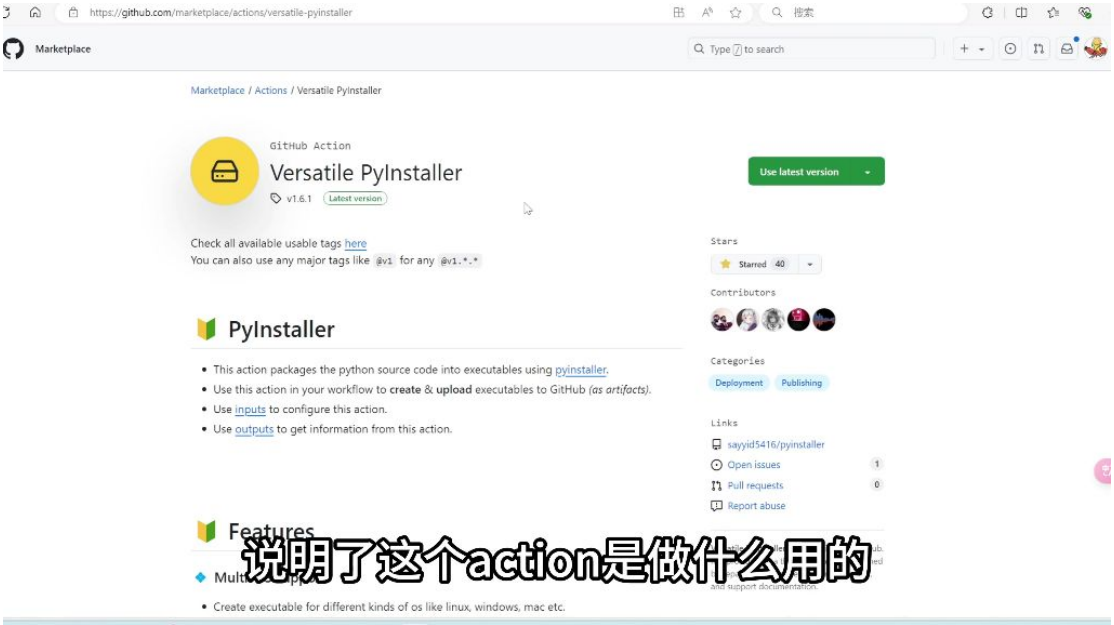
2）Action市场 00:39

- 官方市场 01:15

- 

  - **访问地址**: github.com/actions
  - **特点**:
    - 包含70多个官方维护的常用Action
    - 如checkout action（检出代码到虚拟环境）
    - 每个Action都是独立的脚本文件，存储在代码仓库中
- 社区市场 01:41
- 

  - **访问地址**: github.com/marketplace

- **特点**:
  - 搜索社区开发者共享的Action
  - 可直接复用他人写好的Action
  - 使自动化流程变成Action的组合
- 应用案例 02:06
  - 例题:PyinstallerAction介绍
- 

  - **功能**: 将Python程序打包成可执行文件
  - **使用方法**:
    - 在workflow中使用uses: sayyid5416/pyinstaller@v1
    - 通过with传递参数：
      - python_ver: 指定Python版本
      - spec: 指定入口文件
      - upload_exe_with_name: 指定输出文件名
      - options: 打包选项如--onefile

2. Action的术语 03:11

1）workflow 03:14

-
- **定义**: 工作流程，对应一个YAML文件
- **存储位置**: 存放在仓库的.github/workflows目录
- **层级结构**:
  - 第一层：workflow
  - 第二层：job
  - 第三层：step

2）job 03:25

-

- **特点**:
  - 默认并行运行
  - 每个job运行在独立的虚拟环境中
  - 避免job间相互干扰
- **配置示例**:

```
jobs:
  build:
    runs-on: ubuntu-latest
```

3）step 03:35

- **特点**:
  - 顺序执行
  - 同一job的steps在相同虚拟环境中运行
  - 前一个step的执行结果会影响后一个step
- **示例**:

```
steps:
  - uses: actions/checkout@v4
  - name: Set up Python
    uses: actions/setup-python@v3
```

4）event 04:20

-
- **定义**: 触发workflow执行的事件
- **常见类型**:
  - 代码提交(push)
  - 创建分支/标签
  - 提交PR
  - 定时执行(schedule)
- **配置示例**:

```
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
```

5）应用案例 04:44

- 例题:GitHub Actions工作流程编辑

Edit Preview Code 55% faster with GitHub Copilot Spaces 2 No wrap

```yaml
# This workflow will install Python dependencies, run tests and lint with a single version of Python
# For more information see: https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-python

name: Python application

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

permissions:
  contents: read

jobs:
  build:

    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v4
    - name: Set up Python 3.10
      uses:
      with:
        python-version: "3.10"
    - name: Install dependencies
```

或者往main分支提交pull request的时候

-
  - **典型结构**:
    - name: workflow名称
    - on: 触发事件
    - permissions: 所需权限
    - jobs: 定义的任务
  - **执行流程**:
    - 检出代码(checkout)
    - 设置Python环境
    - 安装依赖
    - 代码规范检查(flake8)
    - 测试(pytest)
  - **虚拟环境**:
    - 通过runs-on指定(如ubuntu-latest/windows-latest)
  - **权限控制**:
    - 通过contents: read设置只读权限

二、知识小结

知识点	核心内容	易混淆点/注意事项	应用示例
GitHub Action 定义	GitHub 的自动化流水线，通过编写 action 脚本实现 CI/CD 等自动化操作	与普通脚本的区别在于**集成到 GitHub 生态**	提交代码后自动运行单元测试、编译、部署
CI/CD 概念	Continuous Integration/Delivery（持续集成/交付）	**CD 可能指持续交付或持续部署**（文中未明确区分）	代码提交 → 测试 → 构建 → 部署全流程自动化
Action 复用机制	开发者可共享独立脚本文件，通过 use标签引用他人编写的 action	版本号需用 Git Tag（如 @v1），**非分支名**	引用 actions/checkout@v3 检出代码

| **核心术语** | | | |

| - Workflow | 对应一个 YAML 文件，存放于 .github/workflows/ | 文件名任意，**扩展名必须为 .yml** | 定义触发条件和执行步骤 |

| - Job | 工作流中的任务单元，默认**并行运行**（独立虚拟机环境） | 与 Step 的层级关系易混淆 | 一个 workflow 包含 build 和 package 两个 job |

| - Step | Job 内的顺序执行步骤，**共享同一虚拟机环境** | 上一步的输出会影响下一步 | 安装依赖 → 代码格式化 → 测试 |

| - Event | 触发 workflow 的事件（如 push、PR、定时任务） | 需明确指定分支（如 on: push: branches: [main]） | 定时任务使用 schedule 事件 |

| **官方资源** | GitHub Actions 市场 提供 70+ 官方 action | actions/checkout 是**最常用**的 action | 检出代码到虚拟机环境 |

| **社区资源** | GitHub Marketplace 搜索社区 action | 需检查 action 的**维护状态和版本** | 使用 pypa/gh-action-pypi-publish 发布 PyPI 包 |

| **实战案例** | | | |

| - Python 项目自动化 | 1. 安装 Python 环境

2. 用 Flake8 格式化

3. 运行 pytest | **需预先配置 requirements.txt** | 示例中删除了未实现的 pytest 步骤 |

| - PyInstaller 打包 | 调用 pypa/gh-action-pypi-publish 生成 exe 文件 | 需指定 Python 版本和入口文件 | 参数：python-version: 3.12, entry_file: main.py |

| **计费规则** | 公共仓库**完全免费**，私有仓库每月 2000 分钟免费额度 | **存储限制 500MB**（仅私有仓库计费） | 查看用量：Settings → Billing → Usage |